SECURITY REMEDIATION · BATCHES 1 & 2

secsuite run 41 — findings & fixes

Scan 4 September 2026 · Remediation 8 September 2026 · **all findings closed**

Target: solanafireplaces.com · shared Next.js app also serving BBQ Grill Outlet and Outdoor Kitchen Outlet

4	4	0	20	3
FINDINGS	FIXED	OUTSTANDING	CHECKS PASSED	BRANDS COVERED

Each finding below is reproduced verbatim from the scan, followed by the fix applied. **Batch 1 covers the three items that needed no product decision** — every one is a response header or a static document, none changes how a page renders or what a visitor can do.

Batch 2 is the clickjacking finding. It was held back from batch 1 because the scan's literal recommendation would have broken a live integration. Confirmation that the integration is retired came on 8 September, and the fix shipped the same day. All four findings are now closed.

Batch 1 — fixed

LOW Missing security header: x-content-type-options — FIXED

`owasp_checks · A05:2021 · CWE-693 · https://solanafireplaces.com/`

> REPORTED
>
> The response does not set the `x-content-type-options` header.
> **Fix:** Send `X-Content-Type-Options: nosniff`.

What was wrong

Without this header a browser may ignore the `Content-Type` we declare and guess from the bytes instead. A file served as `text/plain` can then be sniffed as HTML or JavaScript and executed in our origin.

Fix applied

Added to `next.config.ts` on `source: "/:path*"`, so it applies to every response — pages, API routes and static assets alike, not just HTML.

```
{
  source: "/:path*",
  headers: [
    { key: "X-Content-Type-Options", value: "nosniff" },
    { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
  ],
}
```

No risk of regression: the header does not change what we serve, only forbids the browser from overriding the type we already declare.

LOW Missing security header: referrer-policy — FIXED

`owasp_checks · A05:2021 · CWE-693 · https://solanafireplaces.com/`

> REPORTED
> The response does not set the `referrer-policy` header.
> **Fix:** Send `Referrer-Policy: strict-origin-when-cross-origin`.

What was wrong

With no policy set, the full URL of the page a shopper is on can be sent to every third party we load — analytics, fonts, the chat widget. On a search or filtered listing that URL carries what they were looking for.

Fix applied

Set to the value the scan recommends, in the same rule as above. It sends the full URL to our own origin, the origin only when crossing to another HTTPS site, and nothing when downgrading to HTTP.

This is also the modern browser default. Sending it explicitly means we no longer depend on the browser choosing it for us.

INFO No security.txt — FIXED

`owasp_checks · A05:2021 · CWE-200 · /.well-known/security.txt`

> REPORTED
> No security.txt is published, so security researchers have no documented way to report vulnerabilities.
> **Fix:** Publish /.well-known/security.txt with a Contact field (see securitytxt.org).

What was wrong

A researcher who finds a flaw has nowhere to send it. In practice they either give up or post publicly, and the first we hear of a problem is when someone else does.

Fix applied

New route at `src/app/.well-known/security.txt/route.js`, served as `text/plain; charset=utf-8` per RFC 9116 and cached for a day.

```
Contact: mailto:info@solanafireplaces.com
Expires: 2027-09-08T06:44:58Z
Preferred-Languages: en
Canonical: https://www.solanafireplaces.com/.well-known/security.txt
```

Two decisions inside it

- **A route, not a file in `public/`.** The contact address differs per brand and all three deploy from one codebase, so a static file would publish Solana's address on the BBQ storefront. The address is read from store settings, falling back to the brand's env value.
- **`Expires` is computed, not written down.** RFC 9116 requires the field and treats the file as stale past it. It is set a year ahead on each request, so it cannot silently expire — a hardcoded date would be exactly the thing nobody remembers to update.

Verified

Checked against a running server after the change, on a page, a static asset and an API route — the three response types the rule has to cover.

RESPONSE	X-CONTENT-TYPE-OPTIONS	REFERRER-POLICY
Homepage `/`	nosniff	strict-origin-when-cross-origin
Static asset `/_next/static/…js`	nosniff	strict-origin-when-cross-origin
API route `/api/chat`	nosniff	strict-origin-when-cross-origin
`/.well-known/security.txt`	200, text/plain	contact resolved per brand
Listing page `/fireplaces`	nosniff	strict-origin-when-cross-origin

A clean production build was run after the config change: **compiled successfully, 339/339 static pages**.

> **HSTS was deliberately not added**
>
> The scan did not flag it, and the live site already sends `Strict-Transport-Security: max-age=63072000` from the platform. Emitting a second, weaker value from the application would give browsers the shorter of the two on any route it matched first — a regression dressed as a hardening step.

Batch 2 — fixed

MEDIUM **No clickjacking protection** FIXED

owasp_checks · A05:2021 · CWE-1021 · https://solanafireplaces.com/

> REPORTED
> Neither X-Frame-Options nor a CSP frame-ancestors directive is set, so the page can be embedded in a frame for clickjacking.
> **Fix:** Set `X-Frame-Options: DENY` or `Content-Security-Policy: frame-ancestors 'none'`.

Why this was held back from batch 1

The absence was deliberate, not an oversight. `next.config.ts` emitted `frame-ancestors` only when `_NEXT_ADMIN_FRAME_ANCESTORS` was set, with a comment recording why: the Django admin hosted a store-page configurator that embedded the storefront in an iframe. That variable was set in **no** environment file, so embedding was open to everyone.

Applying the recommendation blindly would have closed the finding and broken that configurator silently — a refused iframe simply renders blank.

Decision taken

The configurator is no longer in use (confirmed 8 September 2026), so the strict option applies: nothing may frame the storefront.

Fix applied

The default inverted — from "open unless configured" to `'none'` unless configured. Both headers are sent, and they are driven by one flag so they can never contradict each other.

```
// CSP
frame-ancestors ${framingAllowed ? `'self' ${adminFrameAncestors.join(" ")}` : "'none'"};

// alongside it, only when framing is not deliberately reopened
...(framingAllowed ? [] : [{ key: "X-Frame-Options", value: "DENY" }]),
```

Three details worth recording

- **Both headers, not one.** The scan accepts either, but `X-Frame-Options` covers browsers predating CSP Level 2. Where both are understood the CSP directive wins.
- **They share a single flag.** `X-Frame-Options` has no allowlist — it can only say DENY or SAMEORIGIN — so if embedding is ever reopened it is omitted entirely and CSP governs alone. Two independent conditions would eventually disagree and block the very embed `frame-ancestors` had just permitted.
- **The escape hatch survives.** Setting `_NEXT_ADMIN_FRAME_ANCESTORS` reopens embedding to `'self'` plus those origins, should an integration return.

Checked first: does anything frame our own pages?

One `<iframe>` exists in the codebase, on a brand page, and it embeds external video. That is `frame-src` — who *we* may embed — and is untouched by `frame-ancestors`, which governs who may embed *us*. Braintree's payment fields are the same case. So `'none'` breaks nothing.

CONFIGURATION	FRAME-ANCESTORS	X-FRAME-OPTIONS
Default (today)	`'none'`	DENY
_NEXT_ADMIN_FRAME_ANCESTORS set	'self' https://be-admin.solanabbqgrills.com	omitted — CSP governs

Both rows verified against a running server, on the homepage and on a listing page, before and after setting the variable.

What the scan already passes

Worth recording, because it is most of the report: **20 checks passed, 2 were not applicable, 3 found issues.** Among those passing are several that commonly fail on a storefront of this age:

- Reflected XSS, and SQL errors from quote injection — both clean
- Session cookies carry HttpOnly, Secure and SameSite; the session ID rotates at login and is never carried in a URL
- Login is rate-limited and does not reveal whether a username exists
- No sensitive files exposed (`.env`, `.git/config`, backups), no source maps served
- CORS is not overly permissive; no mixed content; no open redirect
- HTTP redirects to HTTPS; no server or framework versions disclosed

The gate passed at threshold *high*. Nothing in this batch was urgent; the value is in closing the gap between "no known vulnerabilities" and "hardened by default".